

Utilisation de la plateforme GitHub

GitHub est une plateforme git très populaire.

Elle est facile d'accès, et gratuite pour un usage déjà très intéressant :

- Dépôt git
- Bug tracker
- Reviews / pull-requests
- CI/CD

Plateforme régulièrement utilisée comme espace central par les projets Open Source.

Constitue une forme de réseau social des développeurs Open Source.

- [1. Configuration du compte GitHub](#)
 - [1.1. Anonymisation de l'adresse email](#)
 - [1.2. Configuration d'une clé SSH](#)
- [2. Issues](#)
- [3. Stratégies de merges / reviews / pull-requests](#)
- [4. Forks & pull-requests](#)
- [5. CI/CD](#)
- [6. Annexes](#)
 - [6.1. Documentation utile](#)

1. Configuration du compte GitHub

Après avoir créé un compte GitHub, il convient de faire quelques configurations.

1.1. Anonymisation de l'adresse email

⚠ Risque d'exposition de l'adresse personnelle

Pour pouvoir créer un commit, git impose la configuration d'un nom et d'un email. Cf. [TP - Git base.md](#).

Ces noms et emails sont attachés aux commits, et donc visibles dans l'historique git.

Sur une plateforme en ligne telle GitHub, ce fonctionnement a le défaut d'exposer les adresses emails de manière publique, ce qui n'est pas souhaitable si on veut éviter de se faire spammer.

C'est pourquoi GitHub propose une fonction permettant d'anonymiser son adresse email.

Activer l'anonymisation de l'adresse email :

- Une fois connecté sur la plateforme, cliquer sur l'icône de son profil en haut à droite, puis "Settings" > "Emails".

- "Keep my email address private" : activer l'option.

Relever au passage l'adresse email anonymisée proposée par GitHub.

Exemple : 41943581+alxroyer@users.noreply.github.com.

- En option : "Block command line pushes that expose my email" : activer l'option également.

Cette option permet d'éviter de pousser des commits exposant l'adresse email privée.

! Recipient address rejected

Si on essaie d'envoyer un email à l'adresse anonymisée, l'émetteur de l'email recevra une erreur :

An error occurred while sending mail. The mail server responded: Recipient address rejected

Il reste à renseigner cette adresse email anonymisée dans la configuration git locale.

Dans le dépôt concerné :

```
git config --local user.name "Alexis ROYER"
git config --local user.email "41943581+alxroyer@users.noreply.github.com"
```

1.2. Configuration d'une clé SSH

GitHub impose l'utilisation d'une clé SSH pour pouvoir pousser des commits.

Pour ce faire, suivre la procédure dédiée : <https://docs.github.com/en/authentication/connecting-to-github-with-ssh>.

Si vous n'avez pas déjà créé de clé SSH pour GitHub, il vous faudra certainement en créer une.

Comme la [documentation GitHub](#) le suggère, on utilise la commande `ssh-keygen`.

💡 Astuce : nom clé SSH dédié à l'usage

La première question du menu interactif de `ssh-keygen` concerne le nom de la clé, avec un nom proposé par défaut, du type `~/.ssh/id_ed25519`, ed25519 correspondant au nom de l'algorithme utilisé pour générer le bi-clé clé privée / clé publique.

Ce nom simple a l'avantage d'être automatiquement reconnu par les commandes ssh, rendant la clé directement utilisable.

Mais il reste intéressant de savoir quelle clé SSH correspond à quel usage :

- ne serait-ce que pour s'en souvenir soi-même et de pouvoir faire le ménage le moment venu,
- pour des raisons de sécurité, il est probablement souhaitable d'utiliser des clés différentes pour des usages différents.

Conserver le répertoire proposé par défaut, en l'occurrence `~/.ssh/`. C'est dans ce répertoire qu'on retrouve en général toutes les configurations SSH.

On pourra ensuite choisir un nom de clé du type `id_ed25519_github` pour expliciter que la clé générée sert pour l'authentification avec la plateforme GitHub.

A noter toutefois que si on a spécifié un nom de fichier dédié, la clé n'est pas directement utilisable. SSH doit être configuré.

Pour vérifier que la connexion ssh avec GitHub est fonctionnelle, on pourra utiliser la commande suivante :

```
ssh -T git@github.com
```

Option 1 : Configuration `~/.ssh/config`

Option la plus simple.

Créer ou mettre à jour le fichier `~/.ssh/config`, et déclarer une section :

```
Host github.com
  IdentityFile ~/.ssh/id_ed25519_github
```

Option 2 : Configuration `.git/config`

Dans le cas où on disposerait de plusieurs clés SSH pour GitHub.

On pourra conserver une clé SSH GitHub principale dans le fichier `~/.ssh/config`, puis on pourra sélectionner une clé spécifique dans la configuration git locale du dépôt concerné :

- Cloner dans un premier temps le dépôt en HTTPS (et non en SSH).
- Rentrer dans le dépôt et configurer localement la sélection de la clé :

```
git config --local core.sshCommand "/usr/bin/ssh -i  
~/.ssh/id_ed25519_github_2"
```

On pourra retrouver cette configuration, voire la modifier directement, dans le fichier `.git/config`.

- Changer l'adresse du dépôt `origin` de HTTPS en SSH :

```
git remote -v  
git remote set-url origin git@github.com:alxroyer/edu-sw-dev-  
env.git  
git remote -v
```

Option 3 : Utilisation **ssh-agent**

Sans passer par des configurations, on peut aussi choisir d'utiliser **ssh-agent** et de charger à la volée la clé SSH qu'on veut utiliser à un instant T :

```
# Si `ssh-agent` n'est pas déjà lancé :
eval "$(ssh-agent -s)"
# Lister les clés SSH déjà chargées :
ssh-add -l
# Charger une clé :
ssh-add ~/.ssh/id_ed25519_github
# Décharger une clé :
ssh-add -d ~/.ssh/id_ed25519_github
```

mais il paraît largement plus simple d'utiliser les options de configurations 1 voire 2 présentées avant.

2. Issues

TODO :

- Utilisation du format Markdown
- Démontrer l'intégration croisée entre git et les issues (et les pull-requests)
- Création de *Projects* : <https://docs.github.com/en/issues/planning-and-tracking-with-projects>

3. Stratégies de merges / reviews / pull-requests

TODO :

- Protection de branches
- <https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/configuring-pull-request-merges/about-merge-methods-on-github>
 - Merge commits
 - Squash merging
 - Rebase merging
- *ff-only*:
 - a priori, pas dispo : <https://github.com/orgs/community/discussions/4618>

4. Forks & pull-requests

TODO :

- Cas des contributions Open Source

5. CI/CD

Cf. TP dédié : [GitHub Actions](#).

6. Annexes

6.1. Documentation utile

Gestion des clés SSH :

- Sélection de la clé SSH via `~/.ssh/config` : <https://stackoverflow.com/questions/4565700/how-to-specify-the-private-ssh-key-to-use-when-executing-shell-command-on-git#18839540>
- Sélection de la clé SSH via `ssh-agent` : <https://www.malekal.com/ssh-agent-authentication-cles-ssh-keychain/>
- Troubleshooting "Error: Permission denied (publickey)" sur GitHub : <https://docs.github.com/en/authentication/troubleshooting-ssh/error-permission-denied-publickey>

Stratégies de pull-requests & merges :

- <https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/configuring-pull-request-merges>
- <https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/configuring-pull-request-merges/about-merge-methods-on-github>